

	Gigabit - capable (residential)	Full fibre (residential)	Superfast (residential)	5G coverage from at least one MNOs	WISP FWA	Unable to get decent connection	4G total not spots	Voice and text total not spots
Scotland	77%	62%	96%	91%	2%	0.5%	11%	7%
Northern Ireland	94%	93%	98%	92%	3%	0.2%	2%	1%
Wales	74%	68%	96%	86%	22%	0.5%	5%	3%
England	84%	69%	98%	94%	6%	0.1%	1%	1%

help me normalise data from screenshots above and formulate it into a table like above.

for data from the 2nd to 6th column, normalised value = actual value/maximum value, for example, for scotland 5G coverage, normalised value = 0.91/0.94, where 0.91 is the actual value and 0.94 is the maximum value in that column; for data from 7th to 9th column, they should be normalised by (1-actual value)/(1-minimum value), for example. 4G total not spots for wales = (1-0.05)/(1-0.01)

convert percentage to float values for calculations

	Gigabit - capabl e (reside ntial)	Full fibre (residen tial)	Superfa st (residen tial)	5G coverag e from at least one MNOs	WISP FWA	Unable to get decent connect ion	4G total not spots	Voice and text total not spots
Scotland	77%	62%	96%	91%	2%	0.5%	11%	7%
Northern Ireland	94%	93%	98%	92%	3%	0.2%	2%	1%
Wales	74%	68%	96%	86%	22%	0.5%	5%	3%
England	84%	69%	98%	94%	6%	0.1%	1%	1%

for this set of data, write a simple python code to compute these data into the normalised data you generate later. It should consist of two or more files, one for the mathematics computation, the other one is a txt file or file of other format that contain original information, the txt file will be opened in the mathematical computation code script, and it will output a txt file that contain the normalised data. The files should be extendable (ie: usable if more data inputs are available, such as there are also data of these categories of more regions)

已思考 4s >

Done — I put the files together here:

- [Python script](#)

-

original_broadband_data.csv

	A	B	C	D	E	F	G	H	
1	Region	Gigabit-cap	Full fibre	Superfast	5G coverage	WISP FWA	Unable to get	4G total no	Voice
2	Scotland	77%	62%	96%	91%	2%	0.5%	11%	7%
3	Northern Ir	94%	93%	98%	92%	3%	0.2%	2%	1%
4	Wales	74%	68%	96%	86%	22%	0.5%	5%	3%
5	England	84%	69%	98%	94%	6%	0.1%	1%	1%
6									
7									
8									
9									
10									
11									
12									
13									
14									

Sheet1

normalized_broadband_data.csv

	A	B	C	D	E	F	G	H	
1	Region	Gigabit-cap	Full fibre	Superfast	5G coverage	WISP FWA	Unable to get	4G total no	Voice
2	Scotland	0.8191	0.6667	0.9796	0.9681	0.0909	0.9960	0.8990	0.939
3	Northern Ir	1.0000	1.0000	1.0000	0.9787	0.1364	0.9990	0.9899	1.000
4	Wales	0.7872	0.7312	0.9796	0.9149	1.0000	0.9960	0.9596	0.979
5	England	0.8936	0.7419	1.0000	1.0000	0.2727	1.0000	1.0000	1.000
6									
7									
8									
9									
10									
11									
12									
13									
14									

Sheet1

README_normalization.txt

```
Files included:
- normalize_broadband.py: reads the input CSV and writes
normalized_broadband_data.csv
- original_broadband_data.csv: the source data copied from the
screenshot
- normalized_broadband_data.csv: example output produced from the
source data

How normalization works:
- Columns 2 to 6: actual / max(column)
- Columns 7 to 9: (1 - actual) / (1 - min(column))

To extend the data:
- Add more regions as new rows in original_broadband_data.csv.
```

The script is set up to be extendable: you can add more regions as new rows in the CSV, and it will keep working as long as the column order stays the same. `</>`



Try to give me python code instead of computing them in .csv, you can use .txt or other format to hold input and out put data

已思考若干秒 >

Yes — here is a clean **Python-only** version that reads from a **.txt** file and writes the normalized results to another **.txt** file.

I used **tab-separated text (TSV)** inside the **.txt** files, because it is easy to edit and extend by adding more rows.

1) normalize_broadband.py

</> Python



▶ Run

```
from pathlib import Path
import csv

INPUT_FILE = Path("original_broadband_data.txt")
OUTPUT_FILE = Path("normalized_broadband_data.txt")

# Columns 2 to 6: normalize as actual / max(column)
BENEFIT_COLUMNS = [1, 2, 3, 4, 5] # zero-based indexes in the data columns after Region
# Columns 7 to 9: normalize as (1 - actual) / (1 - min(column))
COST_COLUMNS = [6, 7, 8]

def parse_percent(value: str) -> float:
    """
    Convert percentage text to float.
    Examples:
        '77%' -> 0.77
        '0.5%' -> 0.005
        '0.77' -> 0.77
    """
    value = value.strip()
    if value.endswith("%"):
        return float(value[:-1].strip()) / 100.0
    num = float(value)
    return num if num <= 1 else num / 100.0

def read_tsv(path: Path):
    with path.open("r", encoding="utf-8", newline="") as f:
        reader = csv.reader(f, delimiter="\t")
        rows = [row for row in reader if row and any(cell.strip() for cell in row)]
    return rows

def write_tsv(path: Path, rows):
    with path.open("w", encoding="utf-8", newline="") as f:
        writer = csv.writer(f, delimiter="\t")
        writer.writerows(rows)
```

```
def main():
    rows = read_tsv(INPUT_FILE)

    if len(rows) < 2:
        raise ValueError("Input file must contain a header row and at least one data row.")

    header = rows[0]
    data_rows = rows[1:]

    # Convert metric values to floats
    numeric_rows = []
    for row in data_rows:
        region = row[0]
        values = [parse_percent(x) for x in row[1:]]
        numeric_rows.append([region] + values)

    # Compute column-wise max/min
    metric_columns = list(zip(*[row[1:] for row in numeric_rows]))
    max_values = [max(col) for col in metric_columns]
    min_values = [min(col) for col in metric_columns]

    # Normalize
    output_rows = [header]
    for row in numeric_rows:
        region = row[0]
        values = row[1:]
        normalized = [region]

        for i, actual in enumerate(values):
            if i in BENEFIT_COLUMNS:
                denom = max_values[i]
                norm = actual / denom if denom != 0 else 0.0
            elif i in COST_COLUMNS:
                denom = 1.0 - min_values[i]
                norm = (1.0 - actual) / denom if denom != 0 else 0.0
            else:
                raise ValueError(f"No rule defined for column index {i + 2}")

            normalized.append(f"{norm:.4f}")

        output_rows.append(normalized)

    write_tsv(OUTPUT_FILE, output_rows)
    print(f"Normalized data written to {OUTPUT_FILE}")

if __name__ == "__main__":
    main()
```

2) original_broadband_data.txt

Region	Gigabit-capable	Full fibre	Superfast	5G coverage	WISP	FWA	Unable to get
Scotland	77%	62%	96%	91%	2%	0.5%	11% 7%
Northern Ireland	94%	93%	98%	92%	3%	0.2%	2% 1%
Wales	74%	68%	96%	86%	22%	0.5%	5% 3%
England	84%	69%	98%	94%	6%	0.1%	1% 1%

3) Output file format

The script will create:

normalized_broadband_data.txt

with values like this:

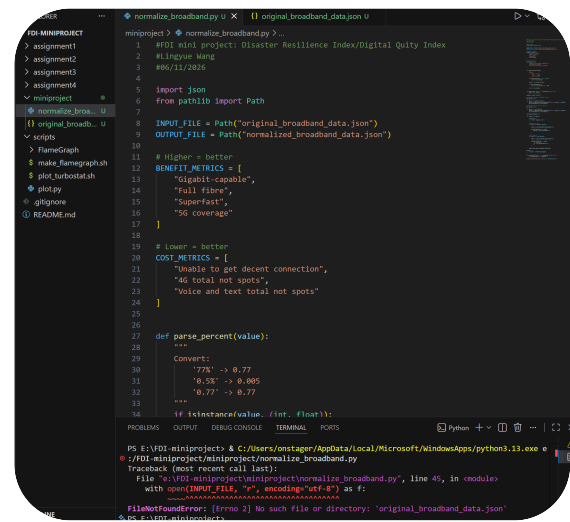
Region	Gigabit-capable	Full fibre	Superfast	5G coverage	WISP	FWA	Unable to get
Scotland	0.8191	0.6667	0.9796	0.9681	0.0909	0.9960	0.8990 0.9394
Northern Ireland	1.0000	1.0000	1.0000	0.9787	0.1364	0.9990	0.9899 1.0000
Wales	0.7872	0.7312	0.9796	0.9149	1.0000	0.9960	0.9596 0.9798
England	0.8936	0.7419	1.0000	1.0000	0.2727	1.0000	1.0000 1.0000

If you want, I can also turn this into a version that reads **JSON** instead of `.txt`, which is even easier to extend when you add more regions or more indicators.









The image shows a VS Code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'FDI-MINIPROJECT' with several files and folders. The code editor shows a Python script named 'normalize_broadband.py' with the following content:

```
1 #FDI mini project: Disaster Resilience Index/Digital Quilty Index
2 #Li Xuyue Xiang
3 #000/11/2026
4
5 import json
6 from pathlib import Path
7
8 INPUT_FILE = Path('original_broadband_data.json')
9 OUTPUT_FILE = Path('normalized_broadband_data.json')
10
11 # Higher = better
12 BENEFIT_METRICS = [
13     "Gigabit-capable",
14     "Full fibre",
15     "Superfast",
16     "5G coverage"
17 ]
18
19 # Lower = better
20 COST_METRICS = [
21     "Unable to get decent connection",
22     "4G total not spots",
23     "Voice and text total not spots"
24 ]
25
26
27 def parse_percent(value):
28     """
29     Convert:
30     '77%' -> 0.77
31     '0.5%' -> 0.005
32     '0.77' -> 0.77
33     """
34
35     if isinstance(value, (int, float)):
36         return value
37
38     # Remove trailing % and convert to float
39     value = value.rstrip('%').strip()
40     return float(value) / 100 if value.endswith('%') else float(value)
```

The terminal window at the bottom shows the command prompt and the error message:

```
PS E:\FDI-miniproject> & C:\Users\onstager\AppData\Local\Microsoft\WindowsApps\python3.13.exe e
E:\FDI-miniproject\miniproject\normalize_broadband.py
Traceback (most recent call last):
  File "e:\fdi-miniproject\miniproject\normalize_broadband.py", line 45, in <module>
    with open(INPUT_FILE, "r", encoding="utf-8") as f:
FileNotFoundError: [Errno 2] No such file or directory: 'original_broadband_data.json'
```


	Gigabit-capable (residential)	Full fibre (residential)	Superfast (residential)	5G coverage from at least one MNOs	Unable to get decent connection	4G total not spots	Voice and text total not spots
Weightage	0.113	0.113	0.113	0.151	0.17	0.17	0.17

After the normalisation, I want to apply a weightage to each normalised category and generate an output file with the weighted normalised data, and for each row (in this case, nation), sum up the weighted normalised values to form an index, this index should be between 0 to 1

the screenshot is the weightage, it should be in a separate json file as well for easy edition

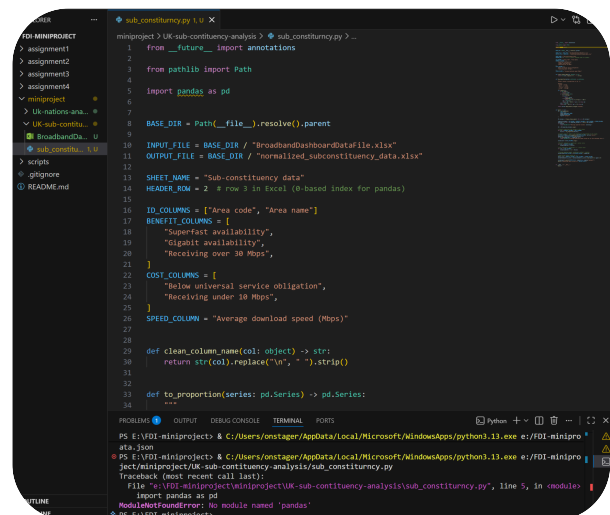
now I have downloaded the data from multiple online sources, and I need you to help me to do the following:
The data size is really big, so please make sure the

ways you choose to analyse the data can handle data size of at least 8000 x 6

The file can be downloaded in excel format, and contains all the data you need to analyse, I want you to write a python code to analyse the data in the excel file I attach, look at data placed in sub-constituency,

File attached: Broadband Data UK Nations - Sub-constituency

Show more ▾



```
sub_constituency.py 1.0 X
miniproject > UK-sub-constituency-analysis > sub_constituency.py > ...
1 from __future__ import annotations
2
3 from pathlib import Path
4
5 import pandas as pd
6
7
8 BASE_DIR = Path(__file__).resolve().parent
9
10 INPUT_FILE = BASE_DIR / "BroadbandDashboardDatafile.xlsx"
11 OUTPUT_FILE = BASE_DIR / "normalized_subconstituency_data.xlsx"
12
13 SHEET_NAME = "Sub-constituency data"
14 HEADER_ROW = 2 # row 3 in Excel (0-based index for pandas)
15
16 ID_COLUMNS = ["Area code", "Area name"]
17 BENEFIT_COLUMNS = [
18     "Superfast availability",
19     "cigabit availability",
20     "Receiving over 30 Mbps",
21 ]
22 COST_COLUMNS = [
23     "Below universal service obligation",
24     "Receiving under 10 Mbps",
25 ]
26 SPEED_COLUMN = "Average download speed (Mbps)"
27
28
29 def clean_column_name(col: object) -> str:
30     return str(col).replace("\n", " ").strip()
31
32
33 def to_proportion(series: pd.Series) -> pd.Series:
34     ...
35
36
37 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
38 PS E:\FDI-miniproject> C:\Users\onstager\AppData\Local\Microsoft\WindowsApps\python3.13.exe e:/FDI-minipro
39 ata.json
40 PS E:\FDI-miniproject> & C:\Users\onstager\AppData\Local\Microsoft\WindowsApps\python3.13.exe e:/FDI-minipro
41 ject/miniproject/UK-sub-constituency-analysis/sub_constituency.py
42 Traceback (most recent call last):
43   File "e:/FDI-miniproject/miniproject/UK-sub-constituency-analysis/sub_constituency.py", line 5, in <module>
44     import pandas as pd
45 ModuleNotFoundError: No module named 'pandas'
46 PS E:\FDI-miniproject>
```


⚠ You may have installed Python packages into your global environment, which can cause conflicts between package versions. Would you like to create a virtual environment with these packages to isolate your dependencies?

Source: Python

Create

Don't show again

```
Installing tzdata-2026.2-py2.py3-none-any.whl (349 kB)
Installing collected packages: tzdata, et-xmlfile, pandas, openpyxl
Successfully installed et-xmlfile-2.0.0 openpyxl-3.1.5 pandas-3.0.3 tzdata-2026.2
PS E:\FDI-miniproject> e:\FDI-miniproject\.venv\Scripts\python.exe -m python -m pip install pandas openpyxl
E:\FDI-miniproject\.venv\Scripts\python.exe: No module named python
PS E:\FDI-miniproject> & e:\FDI-miniproject\.venv\Scripts\python.exe e:\FDI-miniproject\miniproject\UK-sub-c
ontituecy-analysis\sub_constiturncy.py
Traceback (most recent call last):
  File "e:\FDI-miniproject\miniproject\UK-sub-contituecy-analysis\sub_constiturncy.py", line 5, in <module>
    import pandas as pd
ModuleNotFoundError: No module named 'pandas'
PS E:\FDI-miniproject>
```



```
PS E:\FDI-miniproject> e:\FDI-miniproject\.venv\Scripts\python.exe -c "import sys; print(sys.executable)"
E:\FDI-miniproject\.venv\Scripts\python.exe
PS E:\FDI-miniproject> e:\FDI-miniproject\.venv\Scripts\python.exe -c "import pandas; print(pandas.__version__)"
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import pandas; print(pandas.__version__)
    ^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'pandas'
```


Weightage with download speed considered:

	Superfast availability	Gigabit availability	Below universal service obligation	Receiving under 10Mbps	Receiving over 30 Mbps
weights	0.16	0.16	0.26	0.23	0.19

Weightage without download speed considered:

	Superfast availability	Gigabit availability	Below universal service obligation	Receiving under 10Mbps	Receiving over 30 Mbps	Average download speed
weights	0.14	0.14	0.24	0.20	0.18	0.1

Now you need to help me add in weightage calculation
there are two sets of weightages, one with average download speed considered, this other one does not, generate two jsons files separately to hold these two

sets of weightages.

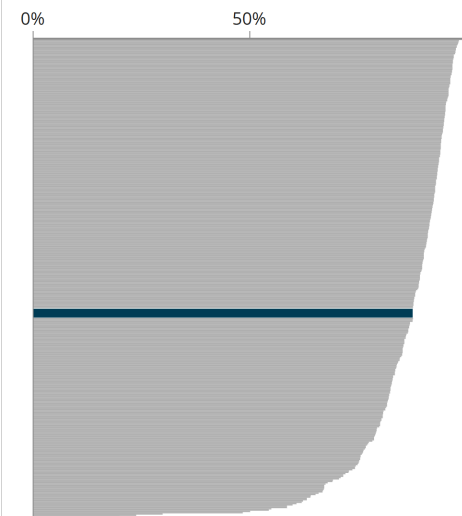
for code, allow the user to choose which weightage they want to use (either leave an apparent line for user to edit in code or use user input to choose); for the one without average download speed weightage, only

Show more ▾

I already have the normalised data, so you do not need to regenerate one, only need to output the weighted parameters and final index for both weightages with and without average speed, edit the code accordingly so it can be run at a faster rate

Gigabit capable broadband

Percentage of premises that have coverage from a gigabit-capable service (download speeds of 1,000 Mbps or more), lower tier/unitary authorities, Jan 2026



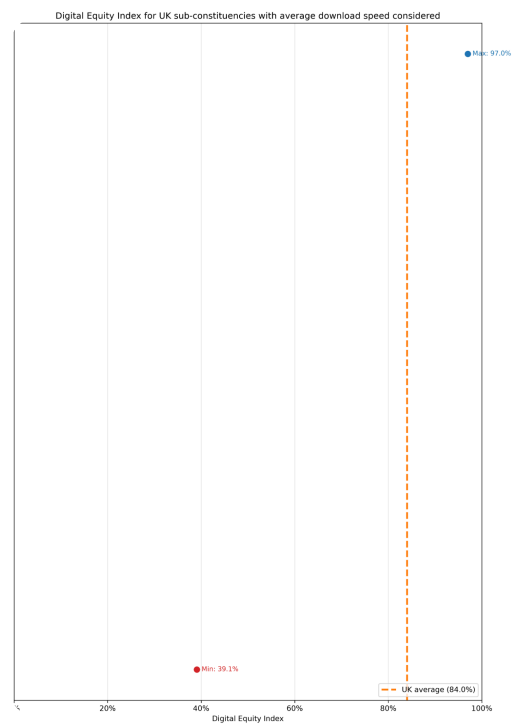
Source: [Ofcom](#)

now your code has worked well, I want you to plot graphs on the two index generated for with and without average download speed in a similar manner as shown in the screenshot I attached.

For the two plots, one should have title "Digital Equity Index for UK sub-constituencies without average download speed considered", the other one should have a title "Digital Equity Index for UK sub-constituencies without average download speed considered". and x-axis should be a scale from 0% to 100%, and the y-axis should be the digital equity

Show more ∨


```
PS E:\FDI-miniproject> ^C
PS E:\FDI-miniproject> & e:/FDI-miniproject/.venv/Scripts/python.exe
e e:/FDI-miniproject/miniproject/UK-sub-constituency-analysis/sub_co
nstituency.py
Traceback (most recent call last):
  File "e:\FDI-miniproject\miniproject\UK-sub-constituency-analysis\
sub_constituency.py", line 10, in <module>
    import matplotlib.pyplot as plt
ModuleNotFoundError: No module named 'matplotlib'
PS E:\FDI-miniproject> █
```



the code generates plots like this, this is not the same as the previous version. I want the plot in general still

look like the previous version and the sample screenshot I sent you, but only need to addin a highlighted line of UK average and label maximum and minimum index data with the name of the sub-constituency, edit your code accordingly

Readme Digital Equity Index

Copy Edit Download

Digital Equity Index

Files in this project

`normalized_subconstituency_data.xlsx`

Contains the sub-constituency data after normalization. All indicator values are already on a common 0–1 scale.

`weights_with_speed.json`

Contains the weights used when average download speed is included in the index.

`weights_without_speed.json`

Contains the weights used when average download speed is excluded from the index.

`weighted_parameters_with_speed.xlsx`

Contains the normalized values, weighted values, and final Digital Equity Index for each sub-constituency using the weight set that includes average download speed.

`weighted_parameters_without_speed.xlsx`

Contains the normalized values, weighted values, and final Digital Equity Index for each sub-constituency using the weight set that excludes average download speed.

`digital_equity_index_with_speed.xlsx`

Contains the final Digital Equity Index values for each sub-constituency using the weight set that includes average download speed.

`digital_equity_index_without_speed.xlsx`

Contains the final Digital Equity Index values for each sub-constituency using the weight set that excludes average download speed.

`digital_equity_index_with_speed.png`

Plot of the Digital Equity Index values for sub-constituencies when average download speed is included. The values are shown in descending order.

`digital_equity_index_without_speed.png`

Plot of the Digital Equity Index values for sub-constituencies when average download speed is excluded. The values are shown in descending order.

What the code does

`Digital_Equity_Index.py` reads `normalized_subconstituency_data.xlsx` and both weight JSON files. It multiplies each normalized indicator by the matching weight, sums the weighted values to form the Digital Equity Index,

and outputs separate Excel files for the version with average speed and the version without average speed.

Two sets of results are produced because the project compares the index with and without average download speed as an indicator. This lets the

已思考 10s >

Done — I drafted the README in the canvas.



I also have an excel called BroadbandDashboardDataFile.xlsx which contain the original unprocessed dataset, include this is readme as well; also add a line in what the code does saying that the reason I use normalised data instead of original data is because using original data and process in code will cause a very slow run of the program, so in the first version of the code I have obtained the normalised data excel already and I use it directly in the current version of code to speed up the program.

Here is the updated README text with the additions you requested:

</> Markdown



```
# Digital Equity Index
```

```
## Files in this project
```

```
### `BroadbandDashboardDataFile.xlsx`
```

```
Contains the original broadband dataset downloaded from external sources. This file includes
```

```
### `normalized_subconstituency_data.xlsx`
```

```
Contains the sub-constituency data after normalization. All indicator values have been trans
```

```
### `weights_with_speed.json`
```

```
Contains the weights used when average download speed is included in the Digital Equity Index
```

```
### `weights_without_speed.json`
```

```
Contains the weights used when average download speed is excluded from the Digital Equity Inc
```

`weighted\_parameters\_with\_speed.xlsx`

Contains the normalized values, weighted values, and Digital Equity Index for each sub-constitu

`weighted\_parameters\_without\_speed.xlsx`

Contains the normalized values, weighted values, and Digital Equity Index for each sub-constitu

`digital\_equity\_index\_with\_speed.xlsx`

Contains the final Digital Equity Index values for each sub-constituency using the weight set

`digital\_equity\_index\_without\_speed.xlsx`

Contains the final Digital Equity Index values for each sub-constituency using the weight set

`digital\_equity\_index\_with\_speed.png`

A plot of the Digital Equity Index values for all sub-constituencies when average download sp

`digital\_equity\_index\_without\_speed.png`

A plot of the Digital Equity Index values for all sub-constituencies when average download sp

What the code does

`Digital\_Equity\_Index.py` reads:

- `normalized\_subconstituency\_data.xlsx`
- `weights\_with\_speed.json`
- `weights\_without\_speed.json`

The code multiplies each normalized indicator by its corresponding weight and sums the weight

The code outputs:

- `weighted\_parameters\_with\_speed.xlsx`
- `weighted\_parameters\_without\_speed.xlsx`
- `digital\_equity\_index\_with\_speed.xlsx`
- `digital\_equity\_index\_without\_speed.xlsx`
- `digital\_equity\_index\_with\_speed.png`
- `digital\_equity\_index\_without\_speed.png`

Two sets of results are produced because the project compares Digital Equity Index values wit

The original dataset (`BroadbandDashboardDataFile.xlsx`) is not processed directly by the cur

This version should be concise enough for a project README while still documenting the full workflow and the reason for separating normalization from index calculation.



where is area code S02002218 in the UK

Area code **S02002218** is not a city or local authority; it is a **Scottish Intermediate Zone (2011 geography)** called **"Isles"** in the **Orkney Islands, Scotland**.

 mapit.mysociety... +1

More specifically:

Attribute	Value
Code	S02002218
Name	Isles
Geography Type	Intermediate Zone (2011)
Local Authority	Orkney Islands
Country	Scotland
Westminster Constituency	Orkney and Shetland

 mapit.mysociety... +1

The zone covers parts of the Orkney archipelago and includes areas such as Burray, South Ronaldsay, and other island communities. Multiple Orkney postcodes are mapped to Intermediate Zone **S02002218 (Isles)**.  获取数据 +2

For your Digital Equity Index project, you can therefore record:

S02002218 → Isles (Orkney Islands, Scotland)



If you have many area codes in your index output and want to automatically convert codes like `s02002218` into readable place names, I can also help you build a lookup table or Python script to batch-translate all Scottish Intermediate Zone codes into names and local authorities.

    ...   Sources

